Resiliencia, HA, Failover y Plan de Pruebas de Rendimiento

Conjunto completo de pruebas para el proyecto, además de herramientas de uso común. El laboratorio de esta carpeta ya cubre ab, Siege, JMeter y fallos de Chaos Monkey a nivel de contenedor.`distributed-three-tier`

1. Comprobaciones de referencia y funcionales (antes de cualquier fallo)

- Cada capa está sana: HAProxy se remonta por todo UP, vuelve bien, Galera, VIPs retenidos por sus amos.`/api/healthwsrep_cluster_size=3`
- Round-robin: los encabezados y rotan entre web1 y 3.`X-Web-NodeX-Presentation-LB`
- Consistencia de datos: una escritura en un nodo Galera es legible en los demás; Los aciertos y errores de caché se comportan correctamente.

-

```
C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s http://localhost/api/health
{"status":"ok"}
C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera1 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e \"SHOW ST
ATUS LIKE 'wsrep_cluster_size'\""
+--------------------+-------+
| Variable_name      | Value |
+--------------------+-------+
| wsrep_cluster_size | 3     |
+--------------------+-------+

C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec presentation-lb-master ip addr show | findstr 172.30.0.250
    inet 172.30.0.250/24 scope global secondary eth0

C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec application-lb-master ip addr show | findstr 172.31.0.250
    inet 172.31.0.250/24 scope global secondary eth0

C:\Users\jesus\Documents\prueba2\distributedsystems-main>for /L %i in (1,1,9) do @curl -s -D - -o NUL http://localhost/api/products/cheapest | f
indstr /i "x-request-trace"
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel (cache miss) -> galera-proxy -> redis-sen
tinel (cache write)
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
```

```
C:\Users\jesus\Documents\prueba2\distributedsystems-main>for /L %i in (1,1,9) do @curl -s -D - -o NUL http://localhost/api/products/cheapest | f
indstr /i "x-request-trace"
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode2 -> application-lb-master -> app3 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app1 -> redis-sentinel -> cache hit
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app2 -> redis-sentinel -> cache hit

C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -X POST http://localhost/api/cache/clear
{"cacheKey":"store:products:cheapest","cacheEntryRemoved":false}
C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -D - -o NUL http://localhost/api/products/cheapest | findstr /i "x-request-trac
e"
x-request-trace: presentation-lb-master -> webnode3 -> application-lb-master -> app2 -> redis-sentinel (cache miss) -> galera-proxy -> redis-sen
tinel (cache write)

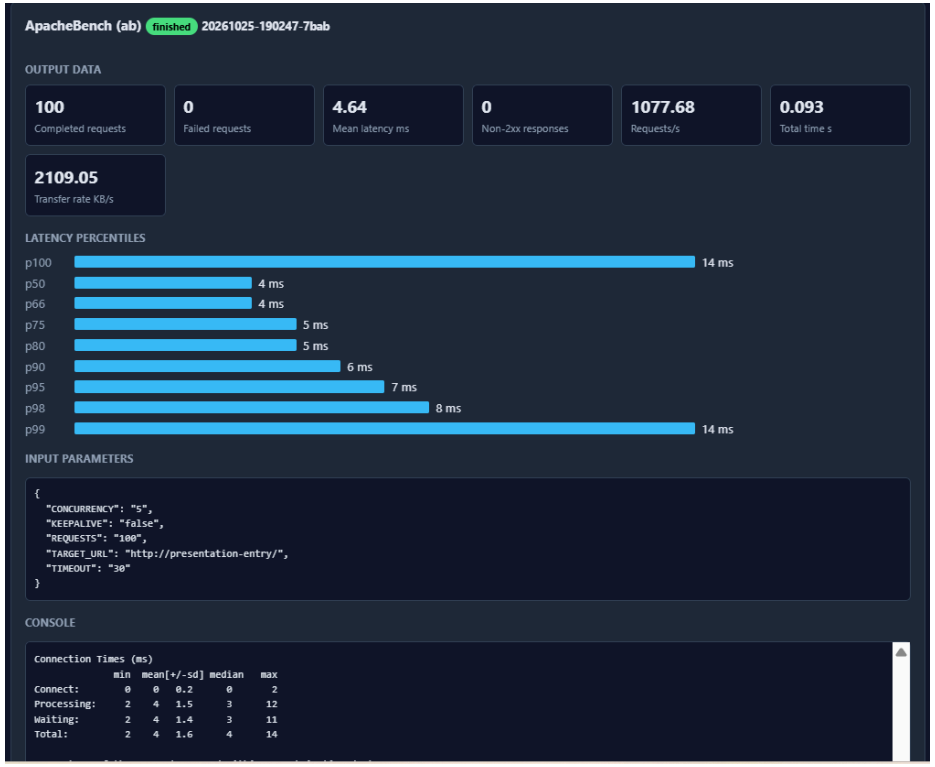
C:\Users\jesus\Documents\prueba2\distributedsystems-main>curl -s -D - -o NUL http://localhost/api/products/cheapest | findstr /i "x-request-trac
e"
x-request-trace: presentation-lb-master -> webnode1 -> application-lb-master -> app3 -> redis-sentinel -> cache hit

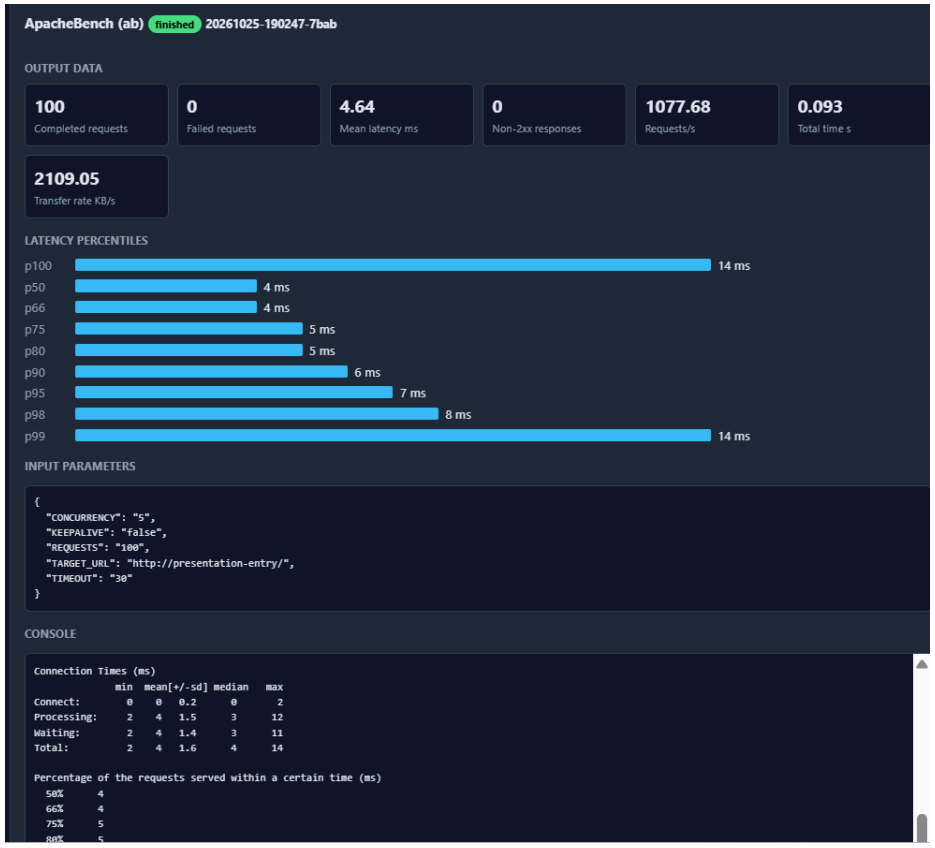
C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera1 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e 'CREATE D
ATABASE IF NOT EXISTS t; CREATE TABLE IF NOT EXISTS t.x(i INT PRIMARY KEY); INSERT INTO t.x VALUES (1);'"

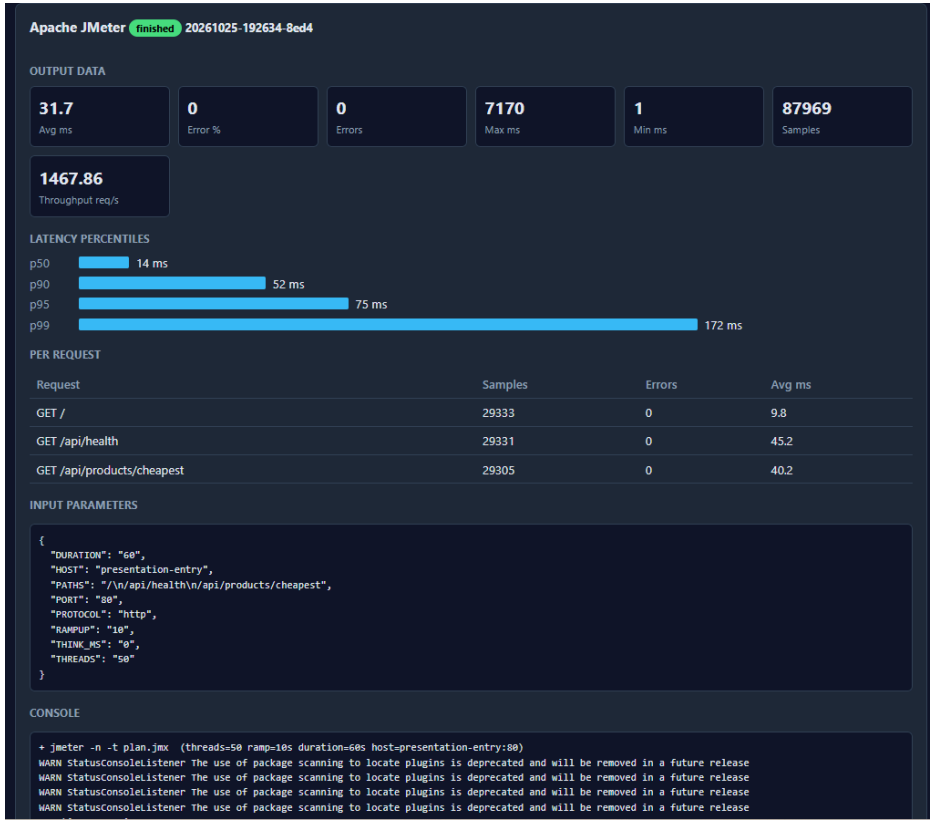
C:\Users\jesus\Documents\prueba2\distributedsystems-main>docker compose exec galera3 sh -c "mariadb -uroot -p$MARIADB_ROOT_PASSWORD -e 'SELECT *
 FROM t.x;'"
+---+
| i |
+---+
| 1 |
+---+
```

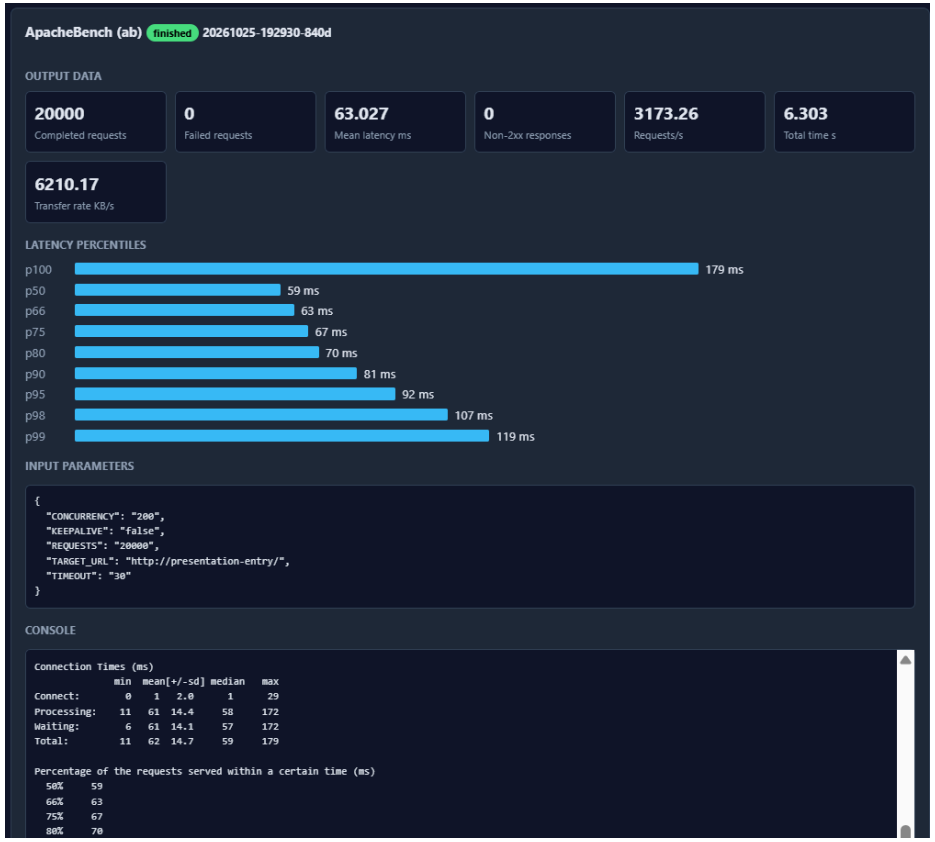
2. Pruebas de rendimiento

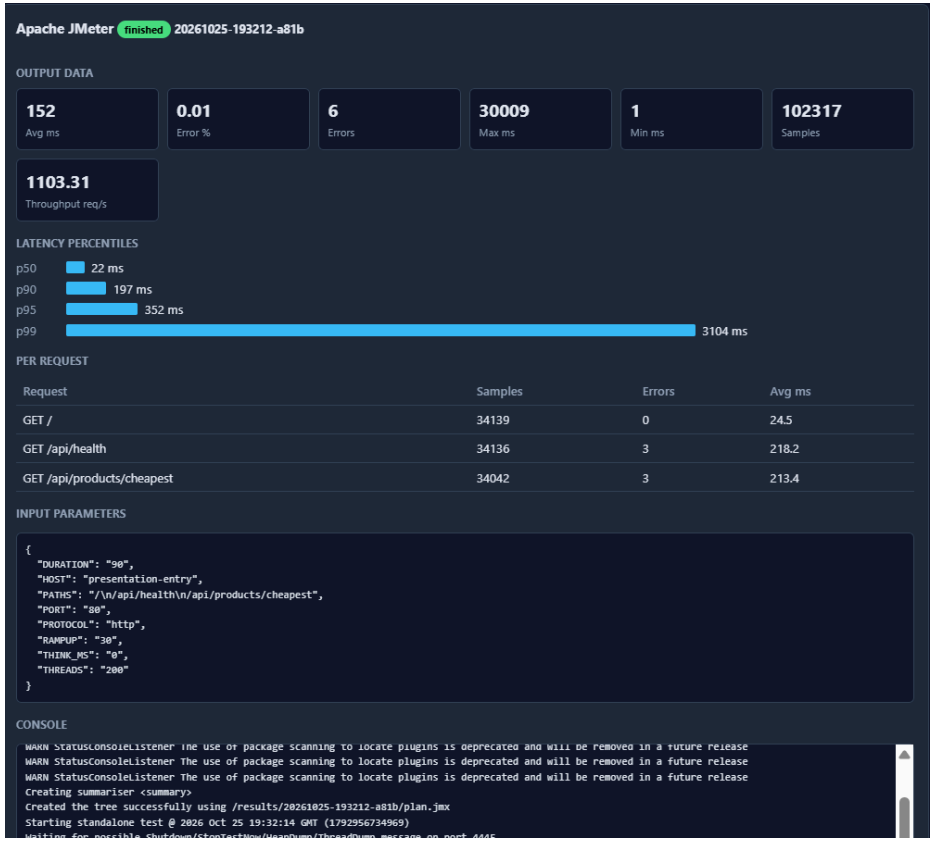
Test	Objetivo	Herramienta de laboratorio	

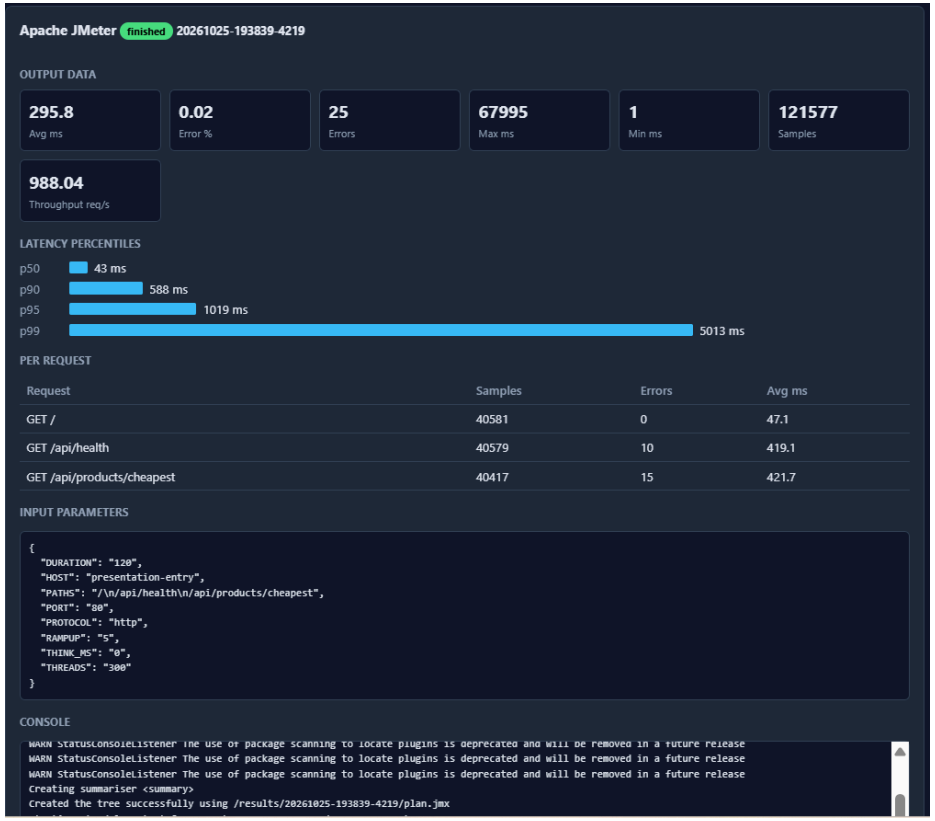
Humo	Confirma que el camino funciona con 1-5 usuarios	ab, JMeter	

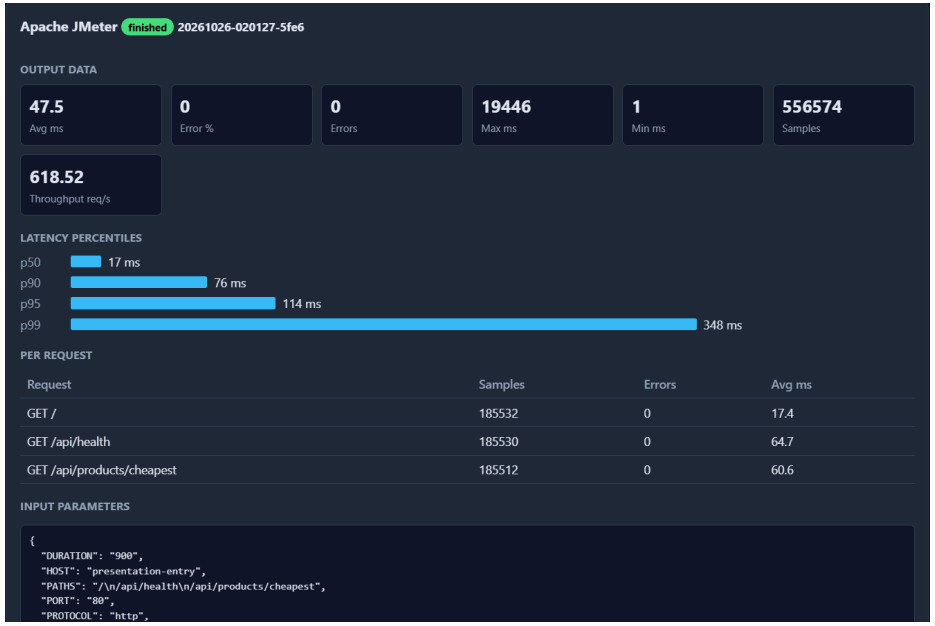

ApacheBench (ab)
finished
20261025-190247-7bab
OUTPUT DATA
100
Completed requests
0
Failed requests
4.64
Mean latency ms
0
Non-2xx responses
1077.68
Requests/s
0.093
Total time s
2109.05
Transfer rate KB/s
LATENCY PERCENTILES
p100 14 ms
p50 4 ms
p66 4 ms
p75 5 ms
p80 5 ms
p90 6 ms
p95 7 ms
p98 8 ms
p99 14 ms
INPUT PARAMETERS
{
"CONCURRENCY": "5",
"KEEPALIVE": "false",
"REQUESTS": "100",
"TARGET_URL": "http://presentation-entry/",
"TIMEOUT": "30"
}
CONSOLE
Connection Times (ms)
min mean[+/-sd] median max
Connect: 0 0 0.2 0 2
Processing: 2 4 1.5 3 12
Waiting: 2 4 1.4 3 11
Total: 2 4 1.6 4 14
Percentage of the requests served within a certain time (ms)
50% 4
66% 4
75% 5

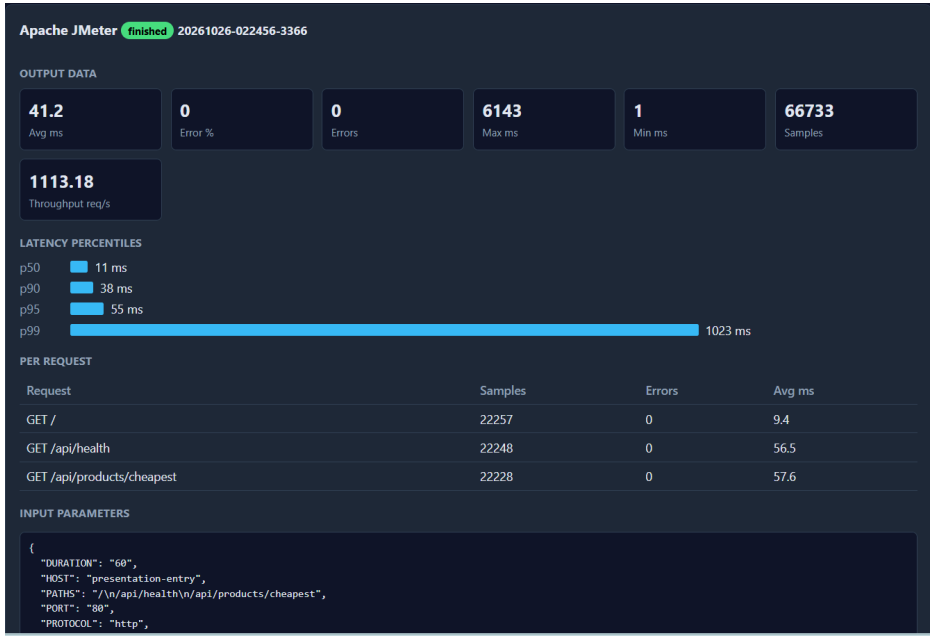
Carga	Tráfico esperado retenido entre 10 y 30 minutos	JMeter, Asedio	

Estrés	Aumenta hasta que los errores o la latencia suban; Encuentra el primer cuello de botella	ab, JMeter	

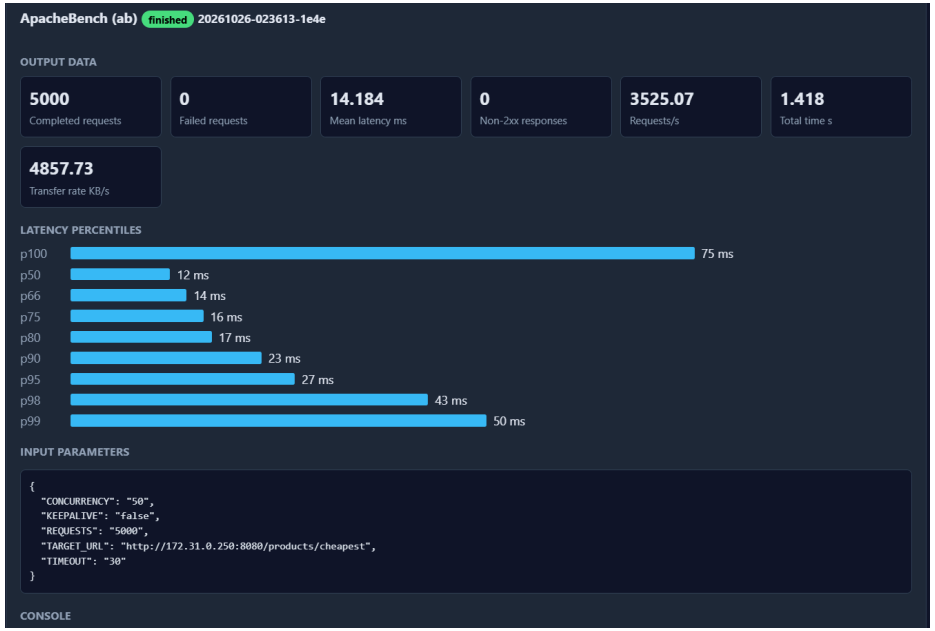

Apache JMeter finished 20261025-193212-a81b
OUTPUT DATA
152
Avg ms
0.01
Error %
6
Errors
30009
Max ms
1
Min ms
102317
Samples
1103.31
Throughput req/s
LATENCY PERCENTILES
p50 22 ms
p90 197 ms
p95 352 ms
p99 3104 ms
PER REQUEST
Request Samples Errors Avg ms
GET / 34139 0 24.5
GET /api/health 34136 3 218.2
GET /api/products/cheapest 34042 3 213.4
INPUT PARAMETERS
{
"DURATION": "90",
"HOST": "presentation-entry",
"PATHS": "/\n/api/health\n/api/products/cheapest",
"PORT": "80",
"PROTOCOL": "http",
"RAMPUP": "30",
"THINK_MS": "0",
"THREADS": "200"
}
CONSOLE
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
Creating summariser <summary>
Created the tree successfully using /results/20261025-193212-a81b/plan.jmx
Starting standalone test @ 2026 Oct 25 19:32:14 GMT (1792956734969)

Spike	Salta de 5 a 300 usuarios en segundos y luego recupérate	JMeter	

Remollo	Carga moderada durante horas para detectar fugas y agotamiento de conexión	JMeter	

Efecto caché	Compara la latencia con y sin Redis (`/cache/clear`)	JMeter	

Capacidad por capa	Prueba web, app y base de datos directamente, saltándose las capas superiores, para localizar el cuello de botella	ab	

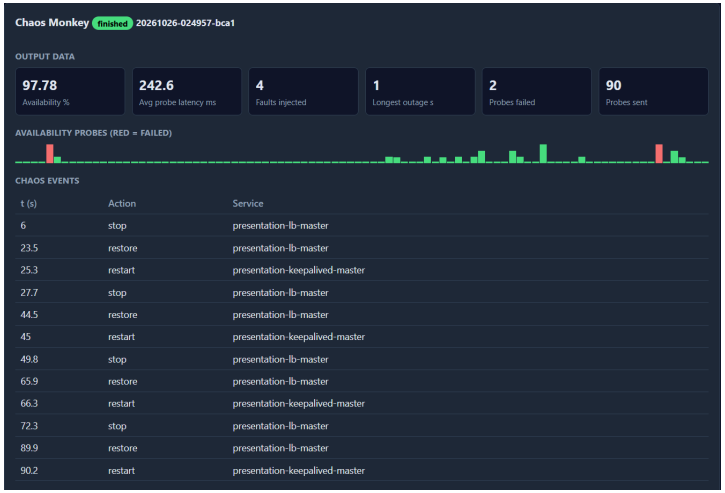

ApacheBench (ab)
finished
20261026-023613-1e4e
OUTPUT DATA
5000
Completed requests
0
Failed requests
14.184
Mean latency ms
0
Non-2xx responses
3525.07
Requests/s
1.418
Total time s
4857.73
Transfer rate KB/s
LATENCY PERCENTILES
p100 75 ms
p50 12 ms
p66 14 ms
p75 16 ms
p80 17 ms
p90 23 ms
p95 27 ms
p98 43 ms
p99 50 ms
INPUT PARAMETERS
{
"CONCURRENCY": "50",
"KEEPALIVE": "false",
"REQUESTS": "5000",
"TARGET_URL": "http://172.31.0.250:8080/products/cheapest",
"TIMEOUT": "30"
}
CONSOLE

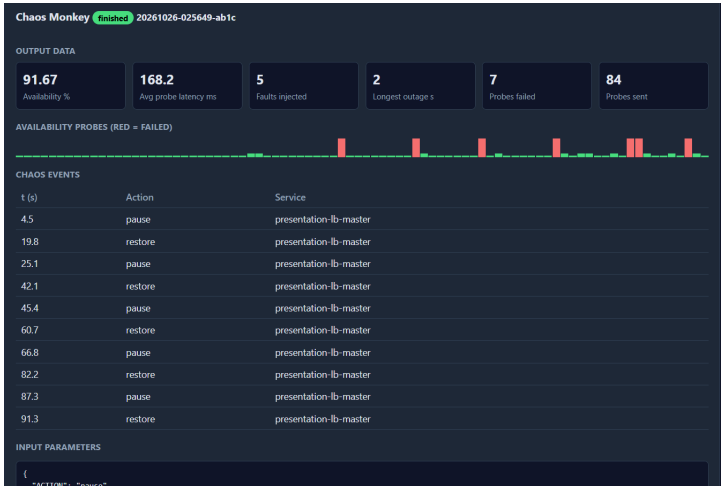
			

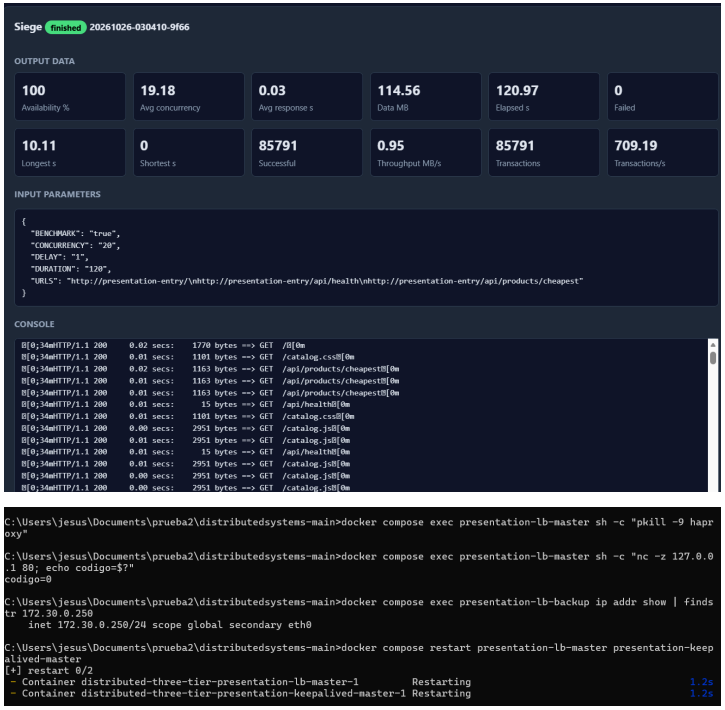
Rastrea la latencia p50/p95/p99, el rendimiento, la tasa de error y la CPU/memoria por contenedor (o cAdvisor). Utiliza objetivos tipo SLO, por ejemplo, p95 < 200 ms y disponibilidad >= 99,9%.`docker stats`

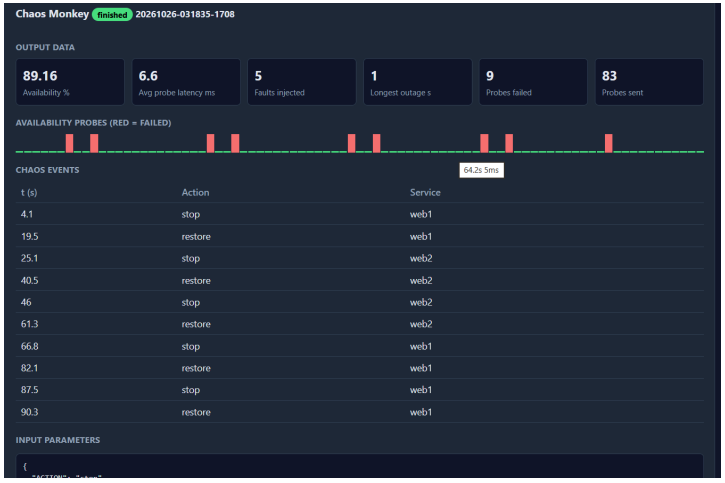
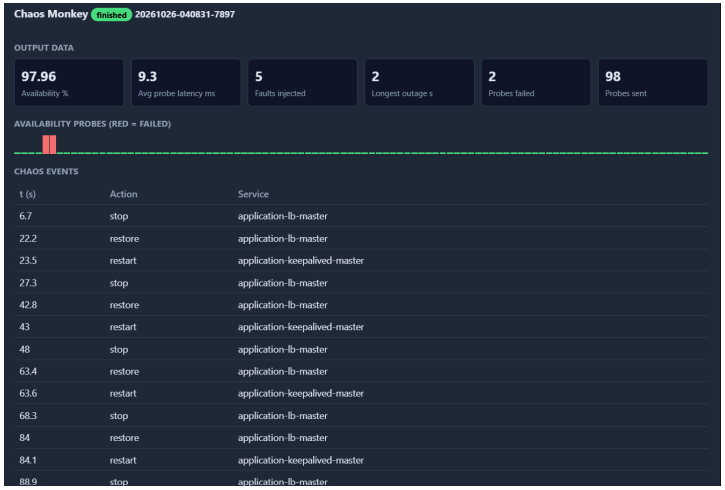
3. Pruebas de HA y fallo

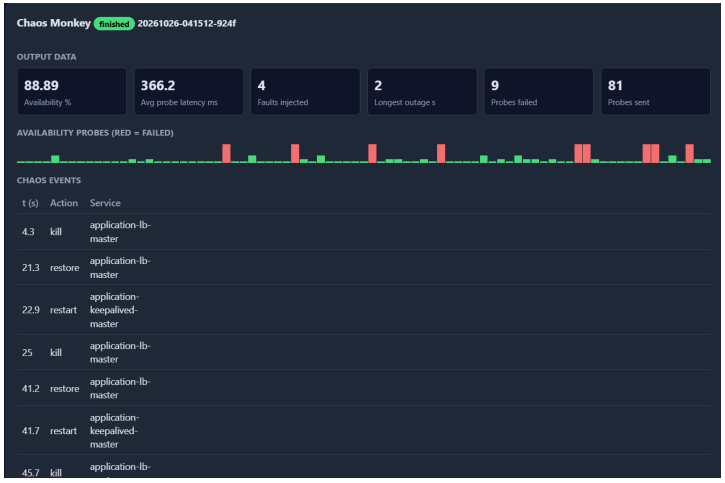
Ejecuta cada uno bajo carga constante; Registrar el tiempo de interrupción y las solicitudes fallidas.

Componente	Falla	Resultado esperado	
Maestro de Presentation LB	Para, mata, pausa	El VIP pasa al suplente en unos 3-4 segundos	 

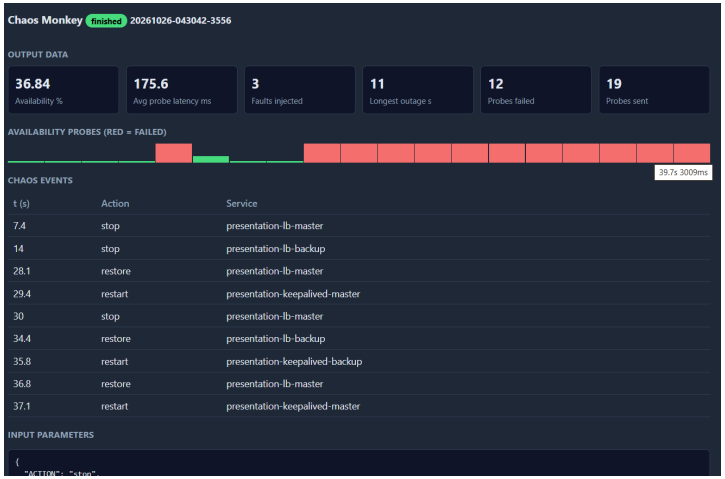

Chaos Monkey
finished
20261026-025649-ab1c
OUTPUT DATA
91.67
Availability %
168.2
Avg probe latency ms
5
Faults injected
2
Longest outage s
7
Probes failed
84
Probes sent
AVAILABILITY PROBES (RED = FAILED)
CHAOS EVENTS
t (s) Action Service
4.5 pause presentation-lb-master
19.8 restore presentation-lb-master
25.1 pause presentation-lb-master
42.1 restore presentation-lb-master
45.4 pause presentation-lb-master
60.7 restore presentation-lb-master
66.8 pause presentation-lb-master
82.2 restore presentation-lb-master
87.3 pause presentation-lb-master
91.3 restore presentation-lb-master
INPUT PARAMETERS
{

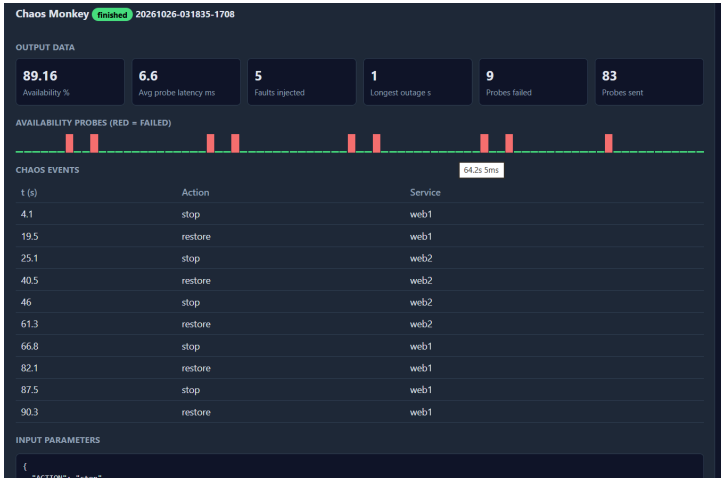
Maestro de Presentation LB	HAProxy cuelga pero el contenedor se queda levantado	El script de comprobación de keepalive falla, la prioridad baja en 20, la copia de seguridad toma el control(puse la prueba que ejecute mientras desconectaba el nodo)	

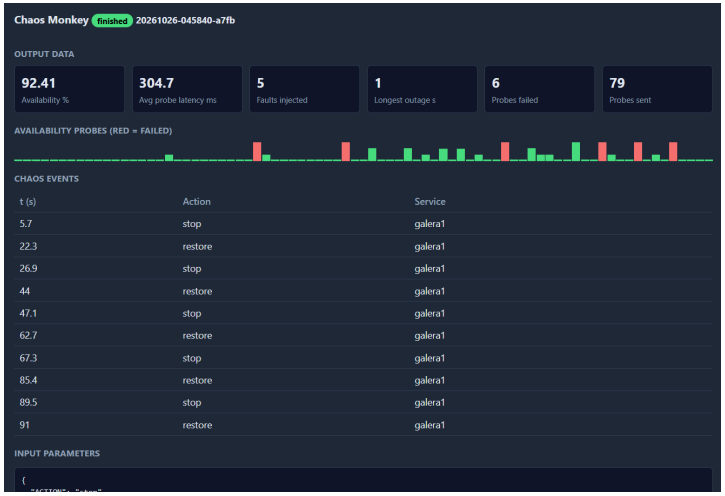
Maestro de Presentation LB	Recuperación	el servicio se mantuvo vivo durante la ejecución sin embargo su desempeño no fue el mejor	
Maestro de aplicación LB	Mismas tres fallas	Igual, en `172.31.0.250`	


Chaos Monkey
finished
20261026-041512-924f
OUTPUT DATA
88.89
Availability %
366.2
Avg probe latency ms
4
Faults injected
2
Longest outage s
9
Probes failed
81
Probes sent
AVAILABILITY PROBES (RED = FAILED)
CHAOS EVENTS
t (s) Action Service
4.3 kill application-lb-master
21.3 restore application-lb-master
22.9 restart application-keepalived-master
25 kill application-lb-master
41.2 restore application-lb-master
41.7 restart application-keepalived-master
45.7 kill application-lb-

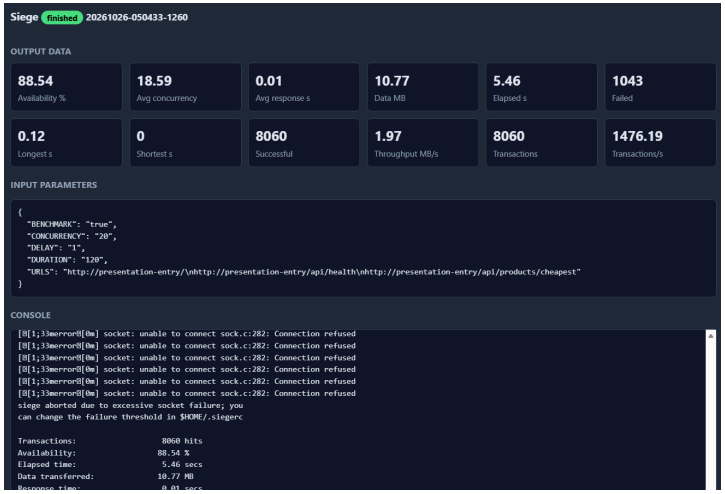

Chaos Monkey
finished
20261026-042236-9d98
OUTPUT DATA
63.64
Availability %
295.8
Avg probe latency ms
4
Faults injected
4
Longest outage s
20
Probes failed
55
Probes sent
AVAILABILITY PROBES (RED = FAILED)
CHAOS EVENTS
t (s) Action Service
3.7 pause application-lb-master
18.8 restore application-lb-master
23.9 pause application-lb-master
39 restore application-lb-master
44.1 pause application-lb-master
59.2 restore application-lb-master
73.3 pause application-lb-master
88.4 restore application-lb-master
INPUT PARAMETERS
{
"ACTION": "pause",
"DOWNTIME": "15",
"DRY_RUN": "false",

Ambos linebackers de un mismo nivel	Para ambos	Corte total, recuperación al reiniciar (test negativo)	
1 de 3 nodos web/app	¡Para la app	El chequeo de salud de HAProxy lo deja caer; Cero o errores cercanos a cero	

2 de 3 nodos web/app	¡Para la web	Capacidad reducida pero aún en servicio	
Nodo Galera	Para Galera2 o Galera3	El clúster mantiene quórum	

			
Nodo Galera	detener Galera1	fracase: es un solo punto`DB_HOST=galera1`	

Redis		Degradación elegante: lecturas más lentas solo de bases de datos, sin 500	
MongoDB		Igual; Críticas degradadas	

Proxy de entrada		Corte total, limitación aceptada de Docker Desktop	

4. Pruebas de ingeniería del caos

Cada experimento: enunciar una hipótesis de estado estacionario (por ejemplo, disponibilidad >= 99% y p95 < 500 ms), inyectar la falla, Compáralo con la hipótesis, recuperar, registrar hallazgos.

- Fallos en contenedores: paradas/muertes/pausas aleatorias, varias víctimas a la vez (cubiertas por el laboratorio).
- Fallos de red: latencia, jitter, pérdida de paquetes, límites de ancho de banda, particiones entre app y base de datos o entre mitades Galera (cerebro dividido). Todavía no en el laboratorio; Usa Pumba o Toxiproxy.
- Fallos de recursos: Consumo de CPU, presión de memoria, disco lleno/lento, agotamiento de descriptores de archivo (, tensión de Pumba).`stress-ng`
- Fallos de tiempo: desfase del reloj, que afecta a Keepalived y Galera.

- Fallas de dependencia: La lentitud de la base de datos sin fallos, exponiendo tiempos muertos perdidos y tormentas de reintentos.
- Fallos compuestos: prueba de carga, fallo del nodo más vaciado de caché al mismo tiempo.
- Fallos de recuperación: reiniciar toda la pila; Revisa el orden de arranque, la caché en frío y el reinicio de Galera tras perder todos los nodos.

5. Comprobaciones de observabilidad y alerta

- Nagios alerta sobre cada fallo dentro de su intervalo de comprobación; Cuenta de alertas perdidas y falsas.
- Logs are sufficient to reconstruct the timeline.
- Record mean time to detect, mean time to recover and error budget burn per experiment.

Gaps the tests will likely expose

- `DB_HOST=galera1` in the app is a single point of failure.
- Redis and MongoDB are single instances.
- `presentation-entry` is a single point of failure.
- Compose does not restart a container that was stopped deliberately.`restart: unless-stopped`

Commonly used tools

Area	Tools
Carga HTTP	k6 (scripts JS, compatible con CI), Locust (Python), Gatling, wrk/wrk2 (tasa constante, latencia precisa), vegeta, hey, bombardier, Artillery
Carga empresarial / distribuida	BlazeMeter, Tsung, NBomber, Taurus (wrapper YAML sobre JMeter, Gatling, etc.)
Caos, contenedores/Docker	Pumba (matar, pausar, netem, estrés), Kit de herramientas del Caos (experimentos declarativos), Gremlin (SaaS)
Caos, Kubernetes	Malla Caos, LitmusChaos, Mono del Caos de Netflix / Ejército Simio (nube, Spinnaker)
Caos, nube	AWS Fault Injection Service, Azure Chaos Studio
Fallos de red	Toxiproxy, TC/Netem, Comcast, iptables

Estrés de huésped/recurso	Tensión-ng, fio, iperf3
Base de datos	sysbench, HammerDB, mysqlslap, pgbench, YCSB, mongoperf, redis-benchmark
Corrección distribuida	Jepsen (para bases de datos como Galera y MongoDB)
Observabilidad	Prometheus + Grafana, cAdvisor, node_exporter, Loki, Jaeger/OpenTelemetry, HAProxy estadísticas y exportador, mysqld_exporter, Nagios/Zabbix
Puerta de CI	Acciones de GitHub con umbrales k6, Taurus
Días de partido	Manuales escritos y autopsias sin culpa

Próximos pasos sugeridos

1. Añade Prometheus y Grafana (exportador HAProxy, cAdvisor) para que cada experimento tenga gráficos de su hipótesis.
2. Añade Pumba o Toxiproxy al laboratorio para experimentos de latencia, pérdida y partición.
3. Añade k6 o Locust para pruebas guionizadas basadas en umbrales que puedan ejecutarse en CI.

4. Elimina el punto único (proxy consciente de la base de datos o endpoint gestionado) y vuelve a ejecutar la prueba de conmutación por error de la base de datos.`galera1`